

VOLUME II

Agentic Design, Workflow and Scoring

A detailed explanation of the multi-agent design patterns, framework choices, role contracts, prompts, state graph, model routing, equations, interfaces, and user journeys.

FOCUS

Why the system is agentic, why its autonomy is bounded, how every node works, and how qualitative research becomes a deterministic portfolio rank.

Implementation snapshot: 02 September 2026

Implemented behavior is separated from planned production extensions on every page.

Contents and reading guide

Read after Volume I. This volume assumes that identity, documents, and accepted claims already exist. It explains how the bounded agent network interprets them and how deterministic services produce the final rank.

Status convention

IMPLEMENTED means executable behavior in this repository. PLANNED means a designed production extension that is catalogued or declared but not wired. BOUNDARY means a deliberate non-goal or limitation.

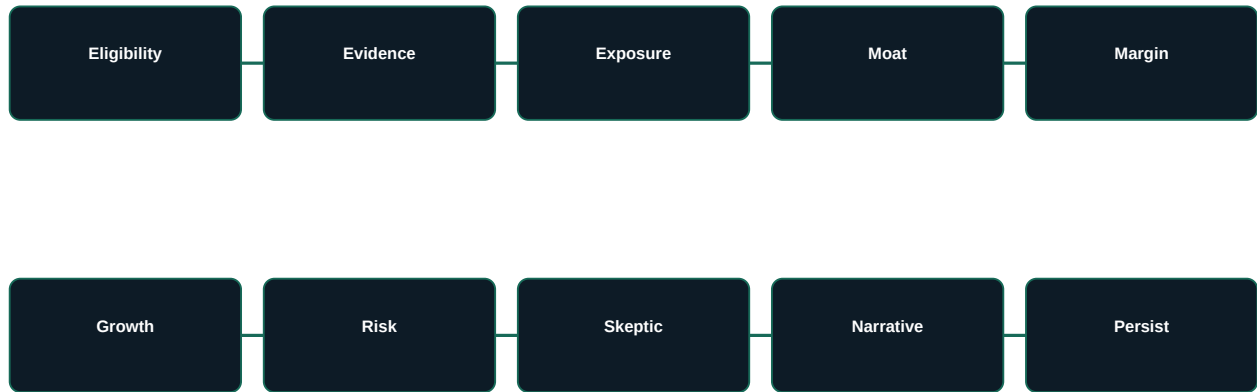
Contents and reading guide	2
Company workflow at a glance	5
What makes this an agentic system	6
Why not one general research agent	7
Agentic design patterns used	8
Pattern: supervisor-worker	9
Pattern: typed pipeline state graph	10
Pattern: evidence ledger as blackboard	11
Pattern: map-reduce portfolio execution	12
Pattern: human-in-the-loop at two boundaries	13
Pattern: critic and evaluator	14
Pattern: deterministic tool core	15
Pattern: bounded autonomy and least privilege	16
Pattern: graceful degradation	17
Pattern: idempotency and immutable reruns	18
Framework decision: LangGraph	19
Why not AutoGen as the primary runtime	20

Why n8n is outside the decision path	21
Framework comparison	22
Model gateway design	23
OpenRouter and Nemotron routing	24
Typed workflow state	25
Research run lifecycle	26
Portfolio fan-out and fan-in	27
Company graph execution trace	28
Company-level failure semantics	29
Agent contract: Eligibility Agent	30
Agent contract: Evidence Agent	31
Agent contract: Exposure Agent	32
Agent contract: Moat Agent	33
Agent contract: Margin Agent	34
Agent contract: Growth Forecast Agent	35
Agent contract: Risk Agent	36
Agent contract: Skeptic Auditor	37
Agent contract: Narrative Agent	38
Common prompt contract	39
Extraction prompt and schema	40
Role-specific moat, growth, and skeptic rubrics	41
Context construction	42
Scoring philosophy	43
Exposure-aware growth conversion	44

Worked exposure example	45
Operating-margin scoring	46
Moat scoring	47
Risk scoring	48
TAFGS worked example	49
Scenario ranking and rank confidence	50
Rankability gates	51
Why capital-stack weights are not score multipliers	52
FastAPI interface	53
Complete API surface	54
CLI and automation surface	55
Primary analyst user journey	56
Data steward user story	57
Methodology owner user story	58
Operator and auditor user stories	59
Quarterly refresh user story	60
Volume II implementation checklist	61
Volume II references	62

Company workflow at a glance

The graph is sequential inside one dossier and parallel across companies. The skeptic is the only node that authorizes final deterministic scoring.



Design rule

Agents interpret and validate bounded evidence. Code calculates, sorts, persists, evaluates, and enforces release gates.

SECTION 01

What makes this an agentic system

IMPLEMENTED

The platform is agentic because bounded roles interpret evidence and update typed state under a supervisor; it is not agentic because a model is allowed to browse and act freely.

Each agent has a decision boundary, a small context, an owned output, and an explicit failure behavior. Agents are composable functions over `WorkflowState`. Their authority is constrained by the workflow order and by deterministic services. Numeric arithmetic, portfolio sorting, persistence, and publication authorization are services rather than conversational agents.

This distinction matters operationally. Model-dependent steps can fail or be replaced without changing the score contract. The same graph runs in offline mode because most roles query already accepted structured claims; only extraction and optional narrative generation require an LLM.

Agentic capability	Implementation
Planning	Predefined typed graph, not open-ended planner
Specialization	Nine role contracts
Tool use	Read-only evidence/model access by role
Memory	Persisted evidence ledger and run state
Critique	Skeptic auditor and contradiction gates
Human control	Evidence acceptance and publication approval

SECTION 02

Why not one general research agent

IMPLEMENTED

A monolithic agent would combine retrieval, interpretation, arithmetic, ranking, and narrative in one opaque context.

The research dimensions have different evidence standards. Margin requires exact Tier-1 reported facts; moat needs component-specific technical and ecosystem evidence; growth needs scenarios and a driver bridge; risk looks for disconfirming evidence; narrative must not introduce new facts. One prompt encourages cross-contamination and makes evaluation ambiguous.

Separate agents create testable contracts. A margin error can be measured as numeric/period mismatch; moat can be scored at component level; skeptic performance can be measured by unsupported-claim recall; narrative can be checked for no-new-facts. Smaller contexts also reduce token cost and prompt-injection exposure.

Why this design

Role separation is justified by different rubrics and permissions, not by naming every function an agent. Deterministic calculations remain plain code because there is no judgment advantage in using an LLM for arithmetic.

SECTION 03

Agentic design patterns used

IMPLEMENTED

The implementation combines patterns rather than relying on a single multi-agent framework abstraction.

Pattern	How it appears	Benefit
Supervisor-worker	ResearchService owns portfolio execution; one graph per company	Bounded coordination and failure isolation
Pipeline/state graph	Fixed ordered nodes over typed state	Reproducibility and inspectability
Blackboard/evidence ledger	Agents share accepted claims and assessment state	Common facts without free-form conversation
Map-reduce	Companies fan out; rankings fan in	Parallelism where dependencies permit
Human-in-the-loop	Proposal acceptance and dossier approval	High-risk judgment remains accountable
Critic/evaluator	Skeptic node tries to falsify and checks gates	Reduces thesis-confirmation bias
Deterministic tool core	Scoring/ranking in code	Exact reproducibility
Graceful degradation	Offline graph and narrative fallback	Useful failure instead of total outage

SECTION 04

Pattern: supervisor-worker

IMPLEMENTED

The outer ResearchService is a supervisor; company graphs are workers with isolated state and deterministic fan-in.

The supervisor validates the cutoff, freezes taxonomy and scoring configuration, selects eligible companies, creates the run, and submits one workflow future per company up to max_workers. Completion order does not affect rank output because assessments are persisted independently and sorting uses score plus company ID tie-break.

A worker exception becomes a non-rankable CompanyAssessment containing only the exception type. Other workers continue. After all futures complete, the supervisor calls the deterministic ranker, saves rankings, updates run status, emits audit events, and optionally generates a report.

Concrete contract

```
SUPERVISOR
freeze run -> submit company graphs -> collect assessments
-> persist failure or success -> deterministic fan-in rank -> complete run
```

Why this design

Portfolio companies are independent until ranking. This pattern gains concurrency without allowing agents to coordinate informally or leak evidence across issuers.

SECTION 05

Pattern: typed pipeline state graph

IMPLEMENTED

A sequential state graph makes prerequisites and stopping behavior explicit.

WorkflowState contains run_id, as_of_date, one company record, that company's eligible claims, the scoring policy, and an assessment under construction. Each node deep-copies the assessment, updates only owned fields, and returns an assessment patch. Credentials, raw document bodies, and other companies never enter state.

The graph order is eligibility, evidence, exposure, moat, margin, growth forecast, risk, skeptic, narrative. LangGraph compiles the same node sequence used by the local fallback. There are no hidden prompt-directed branches; validation errors accumulate and the skeptic decides whether deterministic scoring may run.

Concrete contract

```
eligibility -> evidence -> exposure -> moat -> margin  
-> growth_forecast -> risk -> skeptic_auditor -> narrative
```

Why this design

A fixed graph is appropriate because the research method is known and regulated by policy. Open-ended planning would make it difficult to prove that every company passed the same gates.

SECTION 06

Pattern: evidence ledger as blackboard

IMPLEMENTED

Agents coordinate through accepted, cited claims rather than free-form inter-agent messages.

The evidence_claims table is a durable blackboard. Every agent sees the same company-scoped, point-in-time set and selects claim types relevant to its rubric. Evidence IDs connect outputs to provenance. The assessment object stores derived component values and validation messages, while the claim ledger remains immutable research input.

This design avoids conversational drift, repeated retrieval, and ambiguity about which statement is authoritative. It also permits offline replay: the company graph can run with no internet or model once claims are accepted.

Why this design

A structured blackboard is easier to audit than a transcript. It turns agent collaboration into explicit state transitions and lets deterministic tests reproduce the same decisions.

SECTION 07

Pattern: map-reduce portfolio execution

IMPLEMENTED

The company dimension is mapped in parallel and reduced through deterministic scenario ranking.

Map inputs are frozen company IDs and a shared run configuration. Each map task returns one `CompanyAssessment`. The reduce step filters rankable assessments, sorts base scores descending, creates independent bear and bull orderings, calculates scenario stability, and truncates to the configured Top 20.

The current map executor is a local thread pool. In production, the same boundary should become durable Temporal child workflows or queue jobs with idempotent activity keys. The reduce step must wait for terminal status from all selected companies or apply an explicit partial-run policy.

SECTION 08

Pattern: human-in-the-loop at two boundaries

IMPLEMENTED

Humans intervene where judgment changes the evidence set and where a ranked result becomes an external publication.

Evidence-level review accepts or rejects one validated model proposal after checking the source, exact quote, issuer, date, unit, period, confidence, and rubric. Acceptance creates an EvidenceClaim; rejection preserves the proposal. Dossier-level review approves, rejects, or requests changes for a completed company assessment.

The publication service reads every ranked company's review_status and raises a conflict until all are approved. This two-stage control prevents an apparently polished report from hiding an unreviewed assumption and creates accountability for both inputs and outputs.

Why this design

Human review is most effective at explicit decision gates. Requiring a person to monitor every token or connector call would be expensive without addressing the actual points where model judgment becomes authoritative.

SECTION 09

Pattern: critic and evaluator

IMPLEMENTED

The skeptic agent is a critic node; `evaluate_run` is a deterministic portfolio evaluator.

The skeptic does not improve the thesis. It searches for missing support, source-tier violations, unresolved contradictions, low confidence, low coverage, non-finite values, and incomplete component sets. Only a clean assessment is passed to score calculation.

After portfolio execution, `evaluate_run` recomputes every stored score, calculates coverage, confidence, rankable rate, unsupported-claim count, scenario stability, and approval coverage, then produces research-ready and publication-ready gates. The two evaluators operate at different scopes but share a fail-closed philosophy.

SECTION 10

Pattern: deterministic tool core

IMPLEMENTED

Agents supply validated judgments; tested functions own all arithmetic and ordering.

`operating_margin_score`, `company_ai_driven_cagr`, `weighted_component_score`, `risk_discount`, `create_forecast`, `calculate_assessment_scores`, and `rank_assessments` are deterministic functions. They reject non-finite values, invalid ranges, missing components, unordered scenarios, and invalid discounts.

This design prevents model temperature, provider updates, prompt changes, or narrative wording from changing arithmetic. A run manifest records the formula and policy values, and evaluation recomputes the stored output at 1e-9 tolerance.

Why this design

LLMs are useful for interpretation under ambiguity, not for multiplication, boundary comparison, or sorting. Keeping numbers in code dramatically improves governance and debugging.

SECTION 11

Pattern: bounded autonomy and least privilege

BOUNDARY

The system deliberately narrows what each role can see and do.

Source connectors are read-only and allow-listed. The model gateway can return JSON but has no connector or database tool. Research agents query only the frozen company evidence already supplied in state. The ranker and reporter have no internet access. Publication is a service method guarded by database review state.

Maximum attempts, redirect hops, body bytes, context chunks, proposal count, completion tokens, company workers, and API input ranges are bounded. No agent can trade, send messages, modify external sites, add source adapters dynamically, or expose credentials.

SECTION 12

Pattern: graceful degradation

IMPLEMENTED

Model unavailability does not erase deterministic research capability.

OfflineModelGateway raises a clear error if evidence extraction is attempted, but all accepted-claim agents and scoring remain available. Narrative generation starts with an evidence template; an optional model may replace it only after returning required JSON keys. Any model error appends a warning and keeps the template.

LangGraph is preferred when installed, but the local runner executes the identical ordered agent list. A company failure is isolated and persisted; a portfolio run can still finish with an excluded issuer. These fallbacks are visible in run configuration and assessment warnings.

SECTION 13

Pattern: idempotency and immutable reruns

IMPLEMENTED

Stable hashes and UUIDs make repeated work safe; changed research creates new history.

Raw files are keyed by SHA-256. Documents use a UUID derived from source URL plus content hash. Model proposals use document ID plus normalized proposal JSON. Accepted claims use proposal ID. Company assessments are unique per run/company. Repeating an identical ingestion therefore upserts the same logical record.

A correction, new filing, changed source body, policy update, or new cutoff creates a different document, claim, or research run. Historical rankings are not rewritten to hide change. `compare_runs` makes entry, exit, rank movement, and score movement explicit.

SECTION 14

Framework decision: LangGraph

IMPLEMENTED

LangGraph is used for the bounded company workflow because it represents typed nodes and edges without owning business arithmetic.

StateGraph(WorkflowState) adds each agent as a named node, connects START through the fixed sequence, connects the final node to END, and compiles once for reuse. `invoke(state)` returns the completed assessment. The runtime name is stored in the run configuration snapshot.

The graph supplies inspection, explicit ordering, future checkpointing potential, and a clean path to conditional or parallel nodes. The repository retains a local fallback so core scoring tests do not depend on framework availability.

Why this design

The problem has a known research method and a small typed state. LangGraph fits a governed state machine better than a free-form group chat.

SECTION 15

Why not AutoGen as the primary runtime

BOUNDARY

AutoGen-style conversational multi-agent coordination was considered but not selected for the decision path.

Conversational agents are valuable when planning is open-ended, tools are negotiated dynamically, or multiple specialists must debate an unknown path. Here the role order, evidence inputs, equations, and release gates are known. A chat transcript would add token cost and make it harder to prove that every company received the same procedure.

AutoGen could still be used in a sandboxed research-exploration layer to generate candidate questions or investigate missing evidence. Its outputs would need to enter the same proposal ledger and human-review gate; it should not write accepted claims or ranks directly.

Why this design

Choose the least autonomous framework that still solves the task. The platform needs controlled interpretation, not emergent delegation in the authoritative path.

SECTION 16

Why n8n is outside the decision path

PLANNED

n8n can schedule and notify, but the versioned graph and scoring rules remain in tested Python.

A low-code workflow is useful for calendar triggers, manual approval notifications, ticket creation, or moving final artifacts. It is weaker as the authoritative research engine because visual-node edits can silently change branching, payload shape, retry behavior, or ordering outside the code test suite.

The recommended integration is n8n -> authenticated quarterly-refresh API/CLI -> status polling -> analyst notification. n8n should receive run IDs and summaries, not credentials, raw licensed documents, or authority to update scores.

SECTION 17

Framework comparison

IMPLEMENTED

Framework selection follows the control problem, not popularity.

Option	Best fit	Decision for this platform
LangGraph	Typed, inspectable state workflows	Implemented for company graph
Local Python graph	Offline tests and minimal runtime	Implemented fallback
AutoGen	Conversational multi-agent planning	Optional exploration, not authoritative path
n8n	Triggers, approvals, notifications	Outer automation only
Temporal	Durable distributed long-running workflows	Planned outer supervisor
FastAPI	Typed analyst/operator service	Implemented interface

SECTION 18

Model gateway design

IMPLEMENTED

One interface supports offline, OpenAI-compatible, and local Ollama execution while preserving the same JSON contract.

`complete_json` accepts a system prompt, user prompt, JSON schema, and optional completion cap.

`OpenAICompatibleGateway` normalizes the chat-completions URL, validates HTTPS, sends temperature zero, adds the schema to the user contract, optionally suppresses provider reasoning, and parses one JSON object. Ollama permits plain HTTP only on loopback and requests `format=json`.

The parser handles clean JSON, fenced JSON, or a JSON object embedded in explanatory text, then verifies required top-level keys. Provider/network/schema errors are sanitized as `ModelGatewayError`. Full JSON Schema validation is intentionally not delegated to the gateway; domain validators check individual proposals afterward.

Concrete contract

```
provider = offline | openai_compatible | ollama
complete_json(system_prompt, user_prompt, schema, max_tokens) -> dict
```

SECTION 19

OpenRouter and Nemotron routing

IMPLEMENTED

The OpenAI-compatible route can call OpenRouter's canonical API and a free Nemotron model for bounded public-evidence narrative/proposal work.

Configuration selects provider=openai_compatible, base URL <https://openrouter.ai/api/v1>, a provider model identifier, and an environment-only API key. The gateway appends /chat/completions exactly once, uses temperature zero, enforces a completion ceiling, and can set reasoning effort to none while excluding reasoning from responses.

Free endpoints may be rate-limited, change availability, or apply provider retention policies. They are suitable only for public filings, public web documents, and synthetic data unless organizational review approves confidential processing. The credential must be rotated after disclosure and never appears in prompts, logs, manifests, reports, or source configuration.

Source and reference links

[OpenRouter quickstart](https://openrouter.ai/docs/quickstart)

<https://openrouter.ai/docs/quickstart>

[Nemotron 3.5 Lightning free](https://openrouter.ai/nvidia/nemotron-3.5-lightning:free)

<https://openrouter.ai/nvidia/nemotron-3.5-lightning:free>

SECTION 20

Typed workflow state

IMPLEMENTED

State is intentionally small, company-scoped, and credential-free.

Field	Meaning	Producer/consumer
run_id	Immutable research-run identifier	Supervisor / all nodes
as_of_date	Research clock	Supervisor / prompt and evidence filters
company	Selected issuer/security identity	Database / eligibility and narrative
claims	Accepted eligible EvidenceClaim list	Database / all research roles
policy	Frozen scoring policy	Service / moat, risk, skeptic
assessment	Structured dossier under construction	Every node returns an updated copy

Concrete contract

```
class WorkflowState(TypedDict, total=False):
    run_id: str
    as_of_date: str
    company: dict
    claims: list[EvidenceClaim]
    assessment: CompanyAssessment
    policy: dict
```

SECTION 21

Research run lifecycle

IMPLEMENTED

A run freezes the research clock, selected universe, method versions, model route, and runtime before any company graph starts.

The service validates the date, upserts taxonomy segments, resolves eligible companies, and rejects unknown requested IDs. `create_run` persists created status with taxonomy, scoring, prompt, model, source-policy version, runtime, formula manifest, and selected company IDs. Status moves to running before fan-out.

After all assessments and rankings persist, status becomes completed. An exception at the supervisor boundary changes status to failed with error and audit. Publication is a later explicit transition to published after the review gate. `started_at`, `completed_at`, and `published_at` preserve lifecycle timing.

Concrete contract

```
CREATED -> RUNNING -> COMPLETED -> PUBLISHED  
\  
-> FAILED
```

SECTION 22

Portfolio fan-out and fan-in

IMPLEMENTED

Concurrency is applied across companies, not across dependent steps inside one dossier.

ThreadPoolExecutor uses the configured maximum worker count, clamped to at least one. Futures map back to company IDs. Each completed result is persisted immediately with a `company_assessed` audit event. Failures create a minimal non-rankable assessment and increment `company_workflow_failures_total`.

Fan-in calls `rank_assessments` only after the executor block completes. Because scenario rankings need the whole eligible set, no company can know its final rank inside its own graph. This separation prevents cross-company evidence leakage and keeps portfolio logic deterministic.

SECTION 23

Company graph execution trace

IMPLEMENTED

One company moves through nine roles using only its identity, cutoff-eligible claims, policy, and current assessment.

Step	Consumes	Adds
1 Eligibility	company identity	eligibility and segment
2 Evidence	all claims	IDs, confidence, contradiction warning
3 Exposure	ai_exposure	ratio
4 Moat	six moat claims	components and weighted score
5 Margin	Tier-1 margin	margin and bucket
6 Growth	exposure + scenarios	segment/company forecasts
7 Risk	six severities	discount
8 Skeptic	entire dossier	coverage, gates, score, rankable
9 Narrative	approved dossier	evidence template or model JSON

SECTION 24

Company-level failure semantics

IMPLEMENTED

Validation errors accumulate as data; unexpected exceptions are isolated as workflow failures.

Expected research insufficiency - missing exposure, missing moat component, low confidence, or unresolved contradiction - does not raise an exception. The agent appends a human-readable validation error and the graph continues so the analyst can see all missing dimensions in one dossier.

Unexpected code/provider failures are caught by the portfolio supervisor. The persisted failure reveals the exception type but avoids secret-bearing messages. Other companies continue, and the run reports `excluded_count`. Production should add retry classification and durable activity attempts without changing this fail-closed rankability behavior.

SECTION 25

Agent contract: Eligibility Agent

IMPLEMENTED

Public-company universe eligibility, capital-stack segment capture, and minimum security-master completeness.

Copies the current assessment, reads only the selected company identity, and fails eligibility when the source-derived eligible flag is false or when exchange/security_id is absent.

It does not judge growth quality, valuation, or source evidence. Keeping this gate first prevents research effort and model context from being spent on an invalid security.

Contract field	Definition
Graph node	eligibility
Inputs	company record: eligible, segment, exchange, security_id
Outputs	assessment.eligible, assessment.segment, validation_errors
Failure behavior	The dossier continues through the graph for diagnostics, but final rankability is false because validation errors remain.
Telemetry	agent_eligibility_total plus agent_execution duration/error

Why this design

This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 26

Agent contract: Evidence Agent

IMPLEMENTED

Evidence inventory, tier-adjusted confidence, and early contradiction visibility.

Records every claim ID used by the dossier. If no claims exist it adds a validation error. Otherwise it multiplies each claim confidence by the tier weight (1.00, 0.80, or 0.45) and stores the arithmetic mean.

Contradictory claim IDs are surfaced as warnings here and become a hard rankability error in the skeptic stage. Confidence remains a provenance-quality measure; it is not added to moat or growth.

Contract field	Definition
Graph node	evidence
Inputs	all company claims eligible at the run cutoff
Outputs	evidence_claim_ids, evidence_confidence, warnings/errors
Failure behavior	No as-of-date evidence makes the issuer non-rankable; low confidence is tested against policy later.
Telemetry	agent_evidence_total plus agent_execution duration/error

Why this design

This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 27

Agent contract: Exposure Agent

IMPLEMENTED

Current fraction of issuer revenue directly attributable to AI-factory infrastructure.

Selects the most recent, highest-confidence accepted exposure claim and validates the ratio. It does not fill a missing value from market size, product narrative, or a model guess.

Exposure is separated from segment CAGR so the growth conversion can reflect materiality. A high-growth niche cannot be treated as if it represented the whole issuer.

Contract field	Definition
Graph node	exposure
Inputs	best non-contradictory ai_exposure numeric claim
Outputs	assessment.ai_exposure in [0, 1]
Failure behavior	Missing or out-of-range exposure adds a validation error and prevents final scoring.
Telemetry	agent_exposure_total plus agent_execution duration/error

Why this design
This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 28

Agent contract: Moat Agent

IMPLEMENTED

Six independently evidenced defensibility components and their weighted 0-5 result.

Builds a component dictionary from accepted non-contradictory claims. `weighted_component_score` requires every policy component, validates each score in [0, 5], and normalizes by the configured weight sum.

The agent does not use a single brand-strength intuition. Each component has a separate evidence question so a strong ecosystem cannot conceal weak competitive structure or vice versa.

Contract field	Definition
Graph node	moat
Inputs	architectural lock-in, switching costs, standards/IP, ecosystem/design wins, bottleneck scarcity, competitive-intensity claims
Outputs	moat_components and moat_score
Failure behavior	Any missing or out-of-range component creates an explicit moat-assessment error.
Telemetry	agent_moat_total plus agent_execution duration/error

Why this design

This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 29

Agent contract: Margin Agent

IMPLEMENTED

Reported operating margin and the assignment's normalized 1-5 margin bucket.

Requires regulator/company-primary authority and applies exact boundary handling: >40 scores 5; 30-40 scores 4; 20 to <30 scores 3; 10 to <20 scores 2; below 10 scores 1.

In the SEC path the underlying margin is deterministic OperatingIncomeLoss divided by matching-period annual revenue. Adjusted metrics remain separate.

Contract field	Definition
Graph node	margin
Inputs	best non-contradictory Tier-1 operating_margin claim
Outputs	operating_margin_pct, operating_margin_score, warning for negative margin
Failure behavior	No Tier-1 margin claim or a non-finite value prevents scoring.
Telemetry	agent_margin_total plus agent_execution duration/error

Why this design
This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 30

Agent contract: Growth Forecast Agent

IMPLEMENTED

Bear, base, and bull AI-exposed-business CAGR scenarios and exposure-aware company conversion.

Requires a complete ordered scenario set. create_forecast validates bear <= base <= bull and calls the deterministic exposure conversion for each case.

The scenario claims are analyst-reviewed assumptions supported by disclosed demand, backlog, capacity, product timing, share, and price/mix. The model may propose them but cannot approve or calculate the final score.

Contract field	Definition
Graph node	growth_forecast
Inputs	ai_exposure plus three accepted scenario CAGR claims
Outputs	ScenarioForecast with segment and company-level CAGRs
Failure behavior	Incomplete or unordered scenarios create a validation error.
Telemetry	agent_growth_forecast_total plus agent_execution duration/error

Why this design

This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 31

Agent contract: Risk Agent

IMPLEMENTED

Six risk severities and the capped deterministic discount.

Collects all six 0-5 severities, calculates their weighted mean, divides by five, and multiplies by the 35% maximum discount. The policy owns the weights and cap.

Risk is modeled as a score discount, not a hidden change to growth or moat. This keeps the effect inspectable and supports sensitivity analysis.

Contract field	Definition
Graph node	risk
Inputs	customer concentration, cyclicity, execution, supply chain, geopolitical/regulatory, and commoditization claims
Outputs	risk_components and risk_discount
Failure behavior	Missing or out-of-range risk components prevent scoring.
Telemetry	agent_risk_total plus agent_execution duration/error

Why this design

This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 32

Agent contract: Skeptic Auditor

IMPLEMENTED

Falsification, required-claim coverage, conflict detection, policy gates, and final deterministic score authorization.

Calculates coverage across total revenue, margin, exposure, three scenarios, six moat components, and six risk components. It rechecks Tier-1 authority for financials, blocks any unresolved contradiction, and compares confidence and coverage with policy thresholds.

Only when no validation errors remain does it call `calculate_assessment_scores`. It always sets `review_required=true` and determines rankability without advocating for the thesis.

Contract field	Definition
Graph node	skeptic_auditor
Inputs	complete assessment, all accepted claims, scoring policy
Outputs	citation_coverage, final errors, scores, rankable flag
Failure behavior	Any unresolved evidence or scoring error leaves the dossier persisted but non-rankable.
Telemetry	agent_skeptic_auditor_total plus agent_execution duration/error

Why this design
This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 33

Agent contract: Narrative Agent

IMPLEMENTED

Investor-readable role, catalysts, moat narrative, risks, and contrary evidence from the approved dossier.

Offline mode builds an evidence template from accepted role, catalyst, and contrary-evidence claims. Online mode sends the company-scoped dossier to the model under the common prompt and a strict JSON schema.

The narrative runs after scoring gates and has no authority to alter numeric fields. If the model call or schema fails, the evidence template remains and a warning records the fallback.

Contract field	Definition
Graph node	narrative
Inputs	company identity, completed assessment, accepted evidence, optional model gateway
Outputs	structured narrative with generation_mode
Failure behavior	Model failure degrades gracefully; the research score and evidence stay available.
Telemetry	agent_narrative_total plus agent_execution duration/error

Why this design

This is a separate agent because its rubric, context boundary, and failure meaning are distinct from the other roles. It does not own the final portfolio rank.

SECTION 34

Common prompt contract

IMPLEMENTED

The shared prompt expresses non-negotiable role, time, evidence, injection, citation, abstention, and output rules.

PromptRegistry loads common.txt, substitutes agent_role and as_of_date, and optionally appends one role-specific rubric. The common contract says to make only assigned decisions, use evidence available on/before the cutoff, prefer higher tiers, treat documents as untrusted data, separate fact/calculation/assumption, cite evidence IDs, search for disconfirming evidence, avoid final TAFGS/rank, and abstain when evidence is absent.

Prompts are versioned configuration, not the sole control. The same rules are enforced again by document filtering, schema validation, proposal validators, deterministic scoring, and publication state. The run stores prompt_version so changes can be benchmarked and compared.

Concrete contract

```
ROLE + RESEARCH CLOCK + EVIDENCE POLICY + CONSTRAINTS + OUTPUT
Every document is quoted data; every material claim needs an evidence ID.
```

SECTION 35

Extraction prompt and schema

IMPLEMENTED

The extraction agent proposes evidence; it cannot approve a claim or calculate a ranking.

The user payload contains a small company identity, one document's provenance and injection flags, the matched claim definitions, maximum proposal count, and selected chunks. The output schema requires proposals with `claim_type`, numeric/text value, unit, period, confidence, exact quote, chunk ID, and contradiction flag.

The prompt defines decimal conventions, exact quote behavior, abstention, and the difference between reported fact and forecast/judgment. The schema shapes output, while `_validate_proposal` supplies the authoritative semantic checks after generation.

Concrete contract

```
proposal = {claim_type, value_numeric|null, value_text|null, unit|null,
period_end|null, confidence, evidence_span, page_or_section, contradiction}
```

SECTION 36

Role-specific moat, growth, and skeptic rubrics

IMPLEMENTED

Short rubric files add domain focus without duplicating the common safety contract.

Rubric	Required behavior
Moat	Score six components independently; include supporting and contrary evidence; competitive intensity is defensibility
Growth	Produce bear/base/bull from explicit exposure, demand, backlog, share, price/mix, and capacity drivers
Skeptic	Try to falsify; verify numeric claims, periods, units, and assumptions presented as facts

Why this design

Concise role rubrics are easier to version and evaluate than one giant prompt. Shared safety remains centralized, while domain-specific tests can target each rubric.

SECTION 37

Context construction

IMPLEMENTED

The model receives the minimum context needed for one company, one role, and one decision.

Context includes company/security identity, research cutoff, rubric version, relevant accepted evidence, contrary evidence, and an explicit output schema. Extraction uses top-scoring chunks rather than the whole document. Narrative generation receives the completed company dossier and accepted evidence, not other companies or the entire research lake.

This prevents cross-company leakage, reduces prompt-injection surface, lowers cost, improves provider latency, and makes failed outputs easier to reproduce. Credentials, hidden model reasoning, unrelated analyst notes, and unlicensed raw text are excluded.

SECTION 38

Scoring philosophy

IMPLEMENTED

TAFGS combines defensibility, operating economics, and company-level AI-driven growth, then applies an explicit risk discount.

The original assignment formula is Moat x Operating Margin Score x Forecast AI-Driven Growth. The implementation strengthens it by converting segment CAGR through current exposure and by reporting bear/base/bull scenarios. Risk applies after the base formula and is capped by policy.

Evidence confidence and citation coverage are gates and reporting dimensions, not multipliers. Keeping quality separate avoids rewarding a well-documented weak business or penalizing a strong business through double-counted uncertainty.

Concrete contract

```
base_TAFGS = moat_score * margin_score * company_AI_CAGR_percent  
risk_adjusted_TAFGS = base_TAFGS * (1 - risk_discount)
```

SECTION 39

Exposure-aware growth conversion

IMPLEMENTED

Segment growth is translated into total-company growth by holding non-AI revenue flat solely to isolate the AI contribution.

Let R_0 be current total revenue, e the current AI-factory revenue fraction, g the three-year AI-exposed-business CAGR, and $n=3$ years. Current AI revenue is $R_0 \cdot e$; year-three AI revenue is $R_0 \cdot e \cdot (1+g)^3$. Non-AI revenue stays $R_0 \cdot (1-e)$ for this attribution calculation.

The total-company future revenue ratio is $(1-e) + e \cdot (1+g)^3$. Taking the three-year root and subtracting one gives company AI-driven CAGR. R_0 cancels, so the formula does not require a currency conversion when exposure is already a ratio.

Concrete contract

```
company_AI_CAGR = ((1 - e) + e * (1 + g)^3)^(1/3) - 1
```

Why this design

Without this conversion, a diversified company with 5% exposure would receive the same growth input as a pure-play supplier facing the same segment CAGR.

SECTION 40

Worked exposure example

IMPLEMENTED

A 40% AI segment CAGR does not imply 40% company growth when exposure is only 30%.

Assume total revenue index 100, exposure 0.30, and AI segment CAGR 0.40. AI revenue grows from 30 to 82.32 after three years; non-AI revenue remains 70. Total year-three revenue is 152.32. The annualized company AI-driven CAGR is approximately 15.05%.

The table shows the nonlinear materiality effect. The same segment cycle produces very different company results as exposure changes, which is exactly what the original unadjusted formula misses.

Exposure	AI segment CAGR	Company AI-driven CAGR
5%	40%	2.83%
10%	40%	5.50%
30%	40%	15.06%
50%	40%	23.24%
80%	40%	33.80%
100%	40%	40.00%

SECTION 41

Operating-margin scoring

IMPLEMENTED

Margin is a normalized indicator of pricing power and operating leverage, using exact documented boundaries.

Exactly 40% scores 4, exactly 30% scores 4, exactly 20% scores 3, and exactly 10% scores 2. Negative margins remain score 1 and trigger a warning. Non-finite values raise ScoringError.

The implemented SEC path uses reported GAAP operating income and matching revenue. Adjusted or segment metrics require separate claims and disclosed methodology rather than silent substitution.

Reported operating margin	Score
> 40%	5
30% through 40%	4
20% through <30%	3
10% through <20%	2
<10%	1

SECTION 42

Moat scoring

IMPLEMENTED

Six 0-5 components separate different kinds of defensibility and use versioned weights.

Component	Weight	Evidence focus
Architectural lock-in	22%	Proprietary platform, software dependency
Switching costs	18%	Qualification, migration, installed operations
Standards and IP	18%	Protocol influence, defensible rights, certification
Ecosystem and design wins	18%	Reference designs, partners, named deployment
Bottleneck scarcity	14%	Qualified capacity, lead times, substitutes
Competitive intensity	10%	Favorable structure versus commodity pressure

Concrete contract

```
moat_score = sum(component_i * weight_i) / sum(weights)
```

SECTION 43

Risk scoring

IMPLEMENTED

Risk is a transparent, capped discount derived from six severity components where five is worst.

Risk	Weight
Customer concentration	20%
Cyclicalit	15%
Execution	20%
Supply chain	15%
Geopolitical/regulatory	15%
Commoditization	15%

Concrete contract

```
weighted_severity = sum(severity_i * weight_i) / sum(weights)
risk_discount = weighted_severity / 5 * 0.35
```

Why this design

Applying risk after the growth-quality score keeps the penalty visible. Hiding risk inside moat or growth would make analyst overrides and scenario sensitivity harder to understand.

SECTION 44

TAFGS worked example

IMPLEMENTED

A complete score can be reproduced from four displayed quantities.

Assume moat 4.0, margin score 4, company AI-driven base CAGR 15.05%, and weighted risk severity 2.0/5. The maximum discount is 35%, so risk discount is 14%. Base TAFGS is $4.0 * 4 * 15.05 = 240.8$. Risk-adjusted TAFGS is $240.8 * (1 - 0.14) = 207.09$.

The score is an ordinal research ranking quantity, not a percentage return or fair-value estimate. Comparisons are meaningful only within the same policy version, cutoff, universe, and evidence standard.

Concrete contract

```
base = 4.0 * 4 * 15.05 = 240.80
discount = (2.0 / 5) * 0.35 = 0.14
risk_adjusted = 240.80 * 0.86 = 207.09
```

SECTION 45

Scenario ranking and rank confidence

IMPLEMENTED

Base score determines order; bear and bull ranks reveal sensitivity; confidence combines evidence quality with scenario stability.

The ranker sorts rankable assessments independently by base, bear, and bull risk-adjusted TAFGS. Ties are resolved by `company_id` for deterministic order. Stability equals one minus absolute bear/bull rank spread divided by $\max(1, \text{universe size minus one})$, floored at zero.

Rank confidence is `evidence_confidence` multiplied by $0.65 + 0.35 \times \text{stability}$ and clipped to $[0,1]$. It is reported alongside rank and does not change base order. The Top 20 truncation occurs after the full eligible set receives scenario ranks.

Concrete contract

```
stability = max(0, 1 - abs(bear_rank - bull_rank)/(N-1))
rank_confidence = clip(evidence_confidence * (0.65 + 0.35*stability), 0, 1)
```

SECTION 46

Rankability gates

BOUNDARY

A company is excluded instead of receiving guessed values when required evidence or controls fail.

Implementation details

- Eligible public security with exchange and security ID.
- At least one as-of-date claim.
- Tier-1 total revenue and operating margin.
- AI exposure in $[0,1]$.
- Complete ordered bear/base/bull scenarios.
- All six moat and six risk components in $[0,5]$.
- No unresolved contradictory claim.
- Evidence confidence $\geq 65\%$.
- Required-claim citation coverage $\geq 85\%$.
- All calculations finite and valid.

SECTION 47

Why capital-stack weights are not score multipliers

BOUNDARY

The seed layer shares guide candidate discovery and source effort but do not directly alter TAFGS.

A layer's current share of a reference project is not the same as a supplier's future addressable growth, market share, revenue exposure, or margin. Compute's large spend could already be reflected in growth expectations; construction's smaller share could still produce faster company growth for a focused contractor.

The growth forecast is where end-market demand, product cycle, supplier share, capacity, and project timing belong. Multiplying the capital weight again would double-count opportunity and hard-code one illustrative project mix into every company and geography.

SECTION 48

FastAPI interface

BOUNDARY

The API exposes health, source operations, evidence review, research runs, rankings, evaluations, reports, and audit history.

Pydantic request models bound dates, string lengths, enumerations, result limits, lookback days, and document limits. Long source sync, extraction, and research calls run through `asyncio.to_thread` so the async server loop remains responsive. Domain errors become 422 responses, missing records 404, publication conflicts 409, and database readiness failures 503.

The root workbench, health, readiness, and metrics endpoints are unprotected in the reference service. `/api/v1` operations require the configured X-API-Key. This is a pilot control; production needs SSO, RBAC, user identity in audit events, CSRF/session controls for the UI, and secret-manager integration.

SECTION 49

Complete API surface

IMPLEMENTED

The versioned API is organized around configuration, companies, sources, proposals, runs, review, reports, and evaluation.

Method	Path	Purpose
GET	/health /ready /metrics	Runtime health and process telemetry
GET	/api/v1/config/segments	Taxonomy segments
GET	/api/v1/companies/{id}	Universe, documents, identifiers
GET	/api/v1/sources /source-syncs	Source capability and history
POST	/api/v1/sources/{id}/sync	Bounded connector operation
POST/GET	/api/v1/evidence/extract /proposals	Create/list review-only proposals
POST	/api/v1/evidence/proposals/{id}/review	Accept or reject evidence
POST/GET	/api/v1/runs	Execute/list point-in-time runs
GET	/api/v1/runs/{id}/rankings	Ordered output
GET	/api/v1/runs/{id}/assessments/{company}	Dossiers
POST	/api/v1/runs/{id}/assessments/{company}/reviews	Human dossier decision
POST	/api/v1/runs/{id}/publish	Apply publication gate
POST/GET	/api/v1/runs/{id}/report	Generate/download report
GET	/api/v1/runs/{id}/audit /evaluation	Governance and quality
GET	/api/v1/runs/{previous}/compare/{current}	Quarter-to-quarter diff

SECTION 50

CLI and automation surface

IMPLEMENTED

The CLI exposes the same service methods for local operation, scheduled jobs, and reproducible runbooks.

Group	Commands
Initialize/demo	init-db, seed-demo, reset-demo, approve-ranked-demo
Sources	source-list, source-syncs, source-sync, ingest-sec
Evidence	ingest, extract-evidence, list-proposals, review-proposal
Research	run, quarterly-refresh, list-runs, list-companies, show-run
Governance	review, evaluate, compare-runs, generate-report, publish
Runtime	model-check, serve

Concrete contract

```
aifactory quarterly-refresh --generate-report
aifactory evaluate RUN_ID
aifactory compare-runs PREVIOUS CURRENT
aifactory publish RUN_ID --actor analyst@example.com
```

SECTION 51

Primary analyst user journey

IMPLEMENTED

The analyst moves from source freshness to evidence decisions, ranking review, and controlled publication.

Implementation details

- Choose and freeze the cutoff date.
- Inspect source catalog, credentials readiness, and recent sync failures.
- Sync universe, issuer filings, identity, and discovery sources.
- Review archived documents, parser flags, and claim proposals.
- Accept or reject each evidence proposal with a comment.
- Run selected or complete eligible universe.
- Inspect non-rankable validation errors and Top-20 scenario sensitivity.
- Approve every ranked dossier.
- Publish report and archive manifest.
- Compare with the previous quarter and explain material changes.

SECTION 52

Data steward user story

IMPLEMENTED

As a data steward, I need to add or change a provider without leaking credentials, weakening rights, or changing scoring authority silently.

Acceptance criteria include a disabled catalog entry, documented purpose and evidence tier, environment-only authentication, official endpoint links, bounded transport values, raw-response retention, parser and entity fixtures, licence/retention review, model-processing review, and benchmark results.

The steward can list public capability metadata and see whether a required environment variable is configured, but the API never returns its value. Enablement occurs only after `implementation_status` changes to `implemented` and the code-owned registry contains the adapter.

SECTION 53

Methodology owner user story

IMPLEMENTED

As a methodology owner, I need to change taxonomy, weights, thresholds, prompts, or formulas without silent rank drift.

The owner creates a new version, runs it on a frozen analyst-labelled benchmark, produces score and rank differences, inspects sensitivity by segment and research dimension, and obtains model-risk and research approval. The default changes only after release gates pass.

Every research run stores the taxonomy, scoring, prompt, model, source-policy, runtime, formula manifest, and selected company IDs. Old runs remain reproducible under their original version set.

SECTION 54

Operator and auditor user stories

IMPLEMENTED

Operators watch execution health; auditors reproduce evidence and decisions. Neither role may silently change the score.

Actor	Journey	Evidence of completion
Platform operator	Monitor syncs, logs, metrics, durations, company failures, and scheduled run status	Run/audit records and alerts
Model-risk reviewer	Inspect prompts, schemas, invalid proposals, benchmark drift, and provider changes	Approval record and benchmark report
Auditor	Resolve report hash -> run versions -> assessments -> claim IDs -> document hashes -> raw bytes	Exact score and report reproduction

SECTION 55

Quarterly refresh user story

IMPLEMENTED

The quarterly workflow is a governed research release, not simply a scheduled model call.

The CronJob starts the command at 06:00 UTC on the fifth day of January, April, July, and October with `concurrencyPolicy=Forbid`. The human runbook confirms rights and source health, freezes the cutoff, syncs evidence, validates taxonomy, reviews proposals, executes the graph, evaluates and compares, resolves conflicts, approves the Top 20, and publishes.

A filing-aware event trigger may supplement the calendar. Re-running after a material filing creates a new run and comparison; the existing published result remains intact.

SECTION 56

Volume II implementation checklist

IMPLEMENTED

Use this checklist to verify that agentic behavior remains bounded and reproducible.

Implementation details

- Every agent has one owned decision and typed output.
- Workflow state contains no credentials or cross-company evidence.
- The node order is explicit and versioned.
- Model calls use strict schemas and bounded tokens.
- Evidence proposals require exact quotes and human review.
- Arithmetic and ranking execute only in deterministic code.
- A skeptic node validates coverage, tiers, contradictions, and thresholds.
- Company failures are isolated and persisted.
- Rank confidence remains separate from rank.
- Publication reads human review status from the database.

SECTION 57

Volume II references

IMPLEMENTED

Primary implementation files and framework/provider references used for this volume.

Repository ground truth: src/aifactory/models.py, agents/roles.py, workflow.py, scoring.py, service.py, llm.py, extraction.py, api.py, cli.py, evaluation.py, and reporting.py; config/prompts/*.txt, scoring_policy.json, and extraction_policy.json; docs/agents-and-prompts.md and scoring-methodology.md.

No provider documentation is treated as evidence for company scores. Framework and model links explain runtime integration only.

Source and reference links

[OpenRouter quickstart](https://openrouter.ai/docs/quickstart)

<https://openrouter.ai/docs/quickstart>

[Nemotron 3.5 Lightning free](https://openrouter.ai/nvidia/nemotron-3.5-lightning:free)

<https://openrouter.ai/nvidia/nemotron-3.5-lightning:free>

[LangGraph documentation](https://docs.langchain.com/oss/python/langgraph/overview)

<https://docs.langchain.com/oss/python/langgraph/overview>

[FastAPI documentation](https://fastapi.tiangolo.com/)

<https://fastapi.tiangolo.com/>